```
class Tracking
  constructor: (prefs, http) ->
    @http = http
  start: ->
    @http.listen()

http =
  listen: ->
```

A potential `double` function follows:

```
double = (original) ->
  mock = {}
  for key, value of original
    if value.call?
      do ->
        stub = ->
          stub.called = true
        mock[key] = stub
  mock
```

This `double` function returns a mock version of the original object. It has all the same method names, but the methods themselves are just empty functions that remember if they've been called or not.

In other circumstances your test might call for a spy instead. When you spy on an object, any method calls still occur on the original object but are seen by the spy. A `double` function that returns a spy follows:

```
double = (original) ->
  spy = Object.create original
  for key, value of original
    if value.call?
      do ->
        originalMethod = value
        spyMethod = (args...) ->
          spyMethod.called = true
          originalMethod args...
        spy[key] = spyMethod
  spy
```

Depending on the test framework you're using, there may be both mocking and spying libraries provided for you. The pros and cons of using mocks, spies, and other testing techniques aren't covered here.